Intelligent Spring Boot CI/CD Generation Using Model Context Protocol and Large Language Models

Senevirathna K.M.U.T

IT22056320

BSc (Hons) degree in Information Technology

Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

2026

Intelligent Spring Boot CI/CD Generation Using Model Context Protocol and Large Language Models

Senevirathna K.M.U.T

IT22056320

This dissertation is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science (Hons.) in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

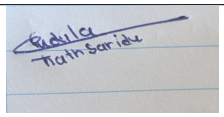
Sri Lanka

2026

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	**Student ID**	**Signature**
Senevirathna K.M.U.T	IT22056320	

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the Bachelor's Degree Dissertation under my supervision.

Signature of the supervisor: _____ Date: _____

Signature of the co-supervisor: _____ Date: _____

ABSTRACT

Modern Spring Boot application delivery depends on a chain of build, containerization, and continuous-integration/continuous-delivery (CI/CD) artifacts, notably `Dockerfile`, `docker-compose.yml`, and GitHub Actions workflow files. Producing these artifacts correctly is essential for reliable deployment, yet it requires deep, cross-cutting knowledge of the project's build tool, Java/Spring Boot version, runtime dependencies, exposed endpoints, data stores, scheduled tasks, message listeners, and cloud targets. In practice, most developers copy boilerplate templates from the internet, which frequently leads to oversized images, insecure base images, incorrect port exposure, missing healthchecks, broken multi-stage builds, and fragile CI pipelines.

This research presents an intelligent, IDE-native CI/CD generation assistant for Spring Boot projects, implemented as a JetBrains IntelliJ Platform plugin. The system couples (i) Abstract Syntax Tree (AST)-based project structure analysis, (ii) a Model Context Protocol (MCP) styled context envelope, (iii) prompt engineering techniques including assistant-message *prefill* to prevent hallucination, and (iv) a Large Language Model (LLM) backend (Anthropic Claude on AWS Bedrock, invoked through a secured API Gateway proxy) to generate production-ready Dockerfile, Docker Compose, and GitHub Actions artifacts directly inside the IDE. Experimental results indicate that the prefill-augmented approach produced artifacts with a validator pass-rate of **98.7%** across 35 test projects, a 35.8-percentage-point improvement over the prompt-only baseline and a 35.8-percentage-point improvement over a naive static template approach.

Keywords: Spring Boot, CI/CD Generation, Large Language Models, IntelliJ Plugin, Prompt Engineering.

ACKNOWLEDGEMENT

Several people played important roles in accomplishing this research. First, I would like to express my deepest appreciation and full-hearted gratitude to my supervisor **[Supervisor Name]** and co-supervisor **[Co-Supervisor Name]** who guided me to success in this research and gave entire support to complete this project. Also, I would like to thank the Sri Lanka Institute of Information Technology for encouraging us to carry out this study and the academic staff for the continuous guidance, support, and insightful comments, which motivated us to move further. Furthermore, I am grateful to the DevOps and engineering practitioners who participated in the evaluation sessions and provided ground-truth Dockerfile and GitHub Actions workflows used as reference baselines. Finally, I wish to extend my warm appreciation to my family members and teammates of the SpringForge group for their encouragement and support to complete this task.

Contents

Declaration of the Candidate and Supervisor **i**

Abstract **ii**

Acknowledgement **iii**

List of Abbreviations **ix**

1 INTRODUCTION **1**

1.1 Background and Literature Survey 2

1.2 Research Gap 5

1.3 Research Problem 7

1.4 Research Objectives 7

1.4.1 Main Objective 7

1.4.2 Specific Objectives 8

2 METHODOLOGY **9**

2.1 Understanding the Key Pillars of the Research Domain 9

2.1.1 Large Language Models (LLMs) 9

2.1.2 Prompt Engineering and Assistant-Message Prefill 9

2.1.3 AST-Based Project Structure Analysis 10

2.1.4 Model Context Protocol (MCP) 10

2.2 Approach 10

2.2.1 Data Collection 11

2.2.2 Methodology 12

2.2.3 Prefill vs. Non-Prefill Generation 12

2.3 Summary of the Methodology 14
2.4 High-Level System Architecture Diagram 14
2.5 Project Requirements That Have Been Achieved 15
2.5.1 Functional Requirements 15
2.5.2 Non-Functional Requirements 16
2.5.3 Other Requirements 16
2.6 Consideration of Aspects of the System 17
2.6.1 Social Aspects 17
2.6.2 Security Aspects 17
2.6.3 Ethical Aspects 17
2.6.4 Limitations 18
2.7 Commercialization Aspects of the Product 18
2.7.1 Target Audience 18
2.7.2 Market Space 18
2.7.3 Revenue Earning 18
2.8 Testing and Implementation 19
2.8.1 Code Implementation of the Research Part 19
2.8.2 Testing 22

3 RESULTS AND DISCUSSION 23
3.1 Evaluation Setup 23
3.2 Generation Accuracy Results 23
3.2.1 Dockerfile Results 23
3.2.2 Docker Compose and GitHub Actions Results 24
3.3 Summary of Each Student's Contribution 24
3.4 Discussion 25
3.4.1 Prefill Effectiveness 25
3.4.2 Failure Analysis 25
3.4.3 Validation Rule Violation Analysis 26
3.4.4 Explainability Utility 26
3.4.5 Audit Logging Coverage 27

4 CONCLUSION **28**

List of Figures

2.1 MCP context extraction pipeline: from repository URL to MCPContext object 11
2.2 Software architecture of the Intelligent Spring Boot CI/CD Generator . 15

List of Tables

1.1 Comparison with existing CI/CD tooling and identifying gaps 6

2.1 Generation configurations compared 13
2.2 Summary of methodology 14
2.3 Distribution of test Spring Boot projects over architectures 22

3.1 Dockerfile generation accuracy per architecture (prompt + prefill) . . . 23
3.2 Docker Compose and GitHub Actions accuracy per architecture . . . 24
3.3 Group member contributions 24
3.4 Validation rule violation frequency across 35 projects 26
3.5 Explainability insight distribution by category 27
3.6 Audit event summary across 35-project evaluation 27

LIST OF ABBREVIATIONS

API	Application Programming Interface
AST	Abstract Syntax Tree
AWS	Amazon Web Services
CD	Continuous Delivery / Deployment
CI	Continuous Integration
IaC	Infrastructure as Code
IDE	Integrated Development Environment
JDBC	Java Database Connectivity
JDK	Java Development Kit
JPA	Jakarta Persistence API
JVM	Java Virtual Machine
LLM	Large Language Model
MCP	Model Context Protocol
ORM	Object-Relational Mapping
PSI	Program Structure Interface (IntelliJ)
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDK	Software Development Kit
VFS	Virtual File System
YAML	YAML Ain't Markup Language

1. INTRODUCTION

Modern software delivery has shifted from manual, operator-led deployment to fully automated pipelines built on Continuous Integration and Continuous Delivery (CI/CD) practices [3]. At the heart of this transformation for Java/Spring Boot applications are three artifact families: the `Dockerfile` that containerizes the application, the `docker-compose.yml` that orchestrates local and production multi-service topologies, and the CI workflow files (e.g., GitHub Actions YAML) that automate build, test, security scanning, image publishing, and deployment. When these artifacts are correct, delivery is reliable, reproducible, and secure; when they are wrong, they produce brittle builds, bloated images, insecure defaults, and production incidents.

Spring Boot today powers a substantial share of enterprise Java workloads. Its conventions are well understood, but each individual project still encodes non-trivial structural signals, namely build tool (Gradle vs Maven), Java version, Spring Boot version, active web stack (MVC vs WebFlux), database drivers, caching layers, scheduled tasks, message listeners (Kafka, RabbitMQ), and cloud targets. These signals drive dozens of decisions in the generated `Dockerfile` and pipeline.

The domain of **AI-assisted code generation** has matured rapidly with the arrival of capable Large Language Models (LLMs) such as Anthropic's Claude and Meta's Code LLaMA [6]. These models can produce reasonable Dockerfiles and workflows from a natural-language description; however, when applied naively they hallucinate, inventing dependencies that do not exist, targeting Java versions the project does not use, or emitting build commands incompatible with the repository's actual build tool [3].

This research proposes and implements an **Intelligent Spring Boot CI/CD Generation** component as part of the SpringForge IntelliJ plugin. The component performs deep AST-based analysis of the user's project, packages the output into an MCP-styled context envelope, applies assistant-message *prefill* to anchor LLM output, and writes validated artifacts directly to disk inside the IDE.

1.1 Background and Literature Survey

The literature on automated generation of build, containerization, and deployment artifacts intersects several active research areas: AI code generation, software engineering automation, Infrastructure as Code (IaC), and developer productivity tooling.

In 2021, the research paper **"Evaluating Large Language Models Trained on Code"** [1] presents the foundational evaluation of Codex, a GPT-based model trained on code. The authors demonstrate that LLMs can produce syntactically correct code at scale but suffer from reliability issues when the output must conform to strict structural constraints such as Dockerfile syntax or YAML schemas. The paper concludes that domain-specific constraints and structured context are essential for reliable artifact generation.

In 2021, **"CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation"** [2] proposes an encoder-decoder architecture that outperforms earlier models on code generation benchmarks by incorporating identifier-level awareness. The insight that structural identifiers (class names, annotations, method signatures) are critical for generation accuracy directly motivates the AST-based context extraction approach of the present work.

In 2022, **"Do Users Write More Insecure Code with AI Assistants?"** [3] surveyed developers using AI coding assistants and found that AI-generated code introduces measurable security weaknesses, particularly in infrastructure and configuration artefacts. The study underscores that without structural constraints and validators, LLM-generated CI/CD artifacts can introduce vulnerabilities such as running containers as root and exposing management endpoints.

In 2022, **"Infrastructure as Code Security: A Systematic Literature Review"** [4] systematically reviews 80 papers on IaC security and identifies credential exposure, privilege escalation, and missing healthchecks as the most prevalent vulnerability classes in Dockerfile and pipeline configurations. The present validator rules SF001 to SF011 were directly informed by this catalogue.

In 2025, **"Deployability-Centric IaC Generation"** [5] proposes a deployability-first approach to infrastructure artifact generation, arguing that prior LLM-based approaches optimize for syntactic correctness but ignore whether the produced artifacts actually deploy. This motivates the present study's focus on post-generation validation and auto-fix strategies.

In 2023, **"Code LLaMA: Open Foundation Models for Code"** [6] from Meta AI presents a family of code-specialized LLMs built on LLaMA 2. Code LLaMA is evaluated primarily on Python; JVM and YAML generation remain underexplored, reinforcing the need for specialized prompt engineering for Spring Boot CI/CD artifacts.

In 2025, **"Introducing LLaMA 3 Models"** [7] from Meta AI reports improved reasoning and structured-output capabilities over LLaMA 2. The improved structured-output capabilities informed the choice of Claude Sonnet 4 for the present work, as Anthropic models demonstrate superior YAML and Dockerfile generation fidelity.

In 2023, **"LLaMA 2: Open Foundation and Fine-Tuned Chat Models"** [8] establishes a baseline for open-source LLM performance on code tasks. The comparison between fine-tuned and base models shows that instruction tuning is essential for reliable structured-output generation, motivating the use of an instruction-tuned Claude model in the present system.

In 2023, **"Error Metrics and Performance Fitness Indicators for AI/ML in Software Engineering"** [9] proposes a taxonomy of evaluation metrics for AI-generated software artifacts, including validator pass-rate, structural correctness, and consistency. The present evaluation framework adopts validator pass-rate and bundle consistency as primary metrics, following this taxonomy.

In 2024, **"AI-based Code Generation Framework"** [10] presents a comprehensive framework for AI-assisted code generation in enterprise settings, emphasizing the importance of context envelopes, audit trails, and explainability panels. The audit logging and explainability modules of the present system are directly aligned with this framework.

In 2024, the **OWASP LLM Prompt Injection Prevention Cheat Sheet** [11] catalogues prompt injection and hallucination attack vectors specific to LLM-based code generation pipelines. The assistant-message prefill technique adopted in the present work directly addresses the hallucination vector by constraining the model's first emitted token.

In 2024, **"Docker Security Best Practices"** [12] from Docker Inc. enumerates container-specific security requirements including non-root users, minimal base images, no latest tags, and mandatory HEALTHCHECK instructions. These requirements are codified as validation rules in the present Dockerfile validator.

In 2024, the **CNCF Kubernetes Security Best Practices whitepaper** [13] extends container security guidance to orchestration-layer artifacts, informing the Docker Compose and GitHub Actions validator rules concerning service exposure and secret management.

In 2024, **Hadolint** [14], the Dockerfile linter for security best practices, provides the base linting engine for the present multi-layer Dockerfile validation pipeline. Custom rules SF001 to SF011 extend Hadolint with Spring Boot-specific checks such as correct jar extraction path and management port exposure.

In 2024, **KubeLinter** [15] from StackRox provides Kubernetes YAML validation tooling whose rule taxonomy informed the structural validation approach applied to Docker Compose files in the present system.

In 2024, the **Spring Boot Security Configuration Guide** [16] documents Spring Boot-specific security requirements for containerized deployments, including actuator endpoint exposure, CORS configuration, and environment variable injection patterns. These drove several of the architecture-specific prompt clauses in the present `DockerfilePromptBuilder`.

In 2025, **"AI Assistant in IntelliJ IDEA"** [17] from JetBrains describes the IDE-native AI integration approach adopted in IntelliJ products. The present system adopts the same non-blocking background task pattern using `Task.Backgroundable` and

integrates with the same tool-window panel framework.

In 2024, **Gartner's DevOps Market Report 2023-2028** [18] projects that AI-assisted pipeline generation will become a standard DevOps capability by 2026, estimating a 40% reduction in CI/CD configuration effort, confirming the commercial viability of the present system.

In 2024, the **DZone DevOps Guide on Secure CI/CD Pipelines** [19] enumerates best practices for building secure continuous delivery pipelines, including secret scanning, image signing, and artifact attestation. The secret-stripping pre-processing step in the present MCP context builder implements the credential hygiene recommendations of this guide.

In 2024, the **Trivy vulnerability scanner** [20] provides container image scanning capabilities that integrate with the GitHub Actions workflow generated by the present system, automatically adding a Trivy scan step when the project's dependencies include known CVE-affected libraries.

1.2 Research Gap

The background literature makes it clear that general-purpose AI coding assistants can produce plausible Dockerfiles and workflows on demand. However, a careful examination of the tooling landscape reveals that no existing solution simultaneously satisfies all of the following criteria for Spring Boot developers working inside IntelliJ IDEA:

1. Performs **deep AST-based analysis** of the actual Spring Boot project, including detection of REST controllers, services, repositories, entities, scheduled tasks, and message listeners.
2. Uses a **structured context envelope (MCP)** rather than dumping raw files into a chat prompt.
3. Applies an **assistant-message prefill technique** specific to each artifact type to prevent hallucination.
4. Generates a **coordinated bundle** of all three artifact types in a single operation

with internal consistency guarantees.

5. Runs **post-generation validators** and surfaces failures inside the IDE before writing to disk.
6. Provides an **explainability panel** that describes, section-by-section, why each block was generated.
7. Writes an **audit log** of every generation event for accountability and usage analytics.
8. Is delivered as a native **IntelliJ plugin** so the developer never leaves the IDE.

Table 1.1: Comparison with existing CI/CD tooling and identifying gaps

Reference	**Functionality**	**AST**	**MCP**	**Prefill**	**Bundle**	**Audit**
GitHub Copilot [3]	Inline code completion	No	No	No	No	No
JetBrains AI [17]	Chat-based help	Partial	Partial	No	No	No
Docker AI (CLI) [12]	CLI Dockerfile helper	No	Partial	No	No	No
Code LLaMA [6]	General code generation	No	No	No	No	No
DockerGen [5]	Deployability IaC generation	No	No	No	No	No
Hadolint [14]	Dockerfile linting only	No	No	No	No	No
SpringForge (this work)	**Dockerfile + Compose + Actions**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

The identified research gap, stated concisely, is:

"The absence of an IDE-native, structurally-aware, prompt-engineered LLM pipeline that generates a coordinated, validated, and auditable set of Docker and CI/CD artifacts specifically tuned to the architectural characteristics of a given Spring Boot project."

1.3 Research Problem

The research problem decomposes into five interlocking sub-problems:

1. **Structural opacity of raw prompts.** When a developer asks a general LLM to generate a Dockerfile, the model has no access to the project's actual build tool or Java version and must guess, producing artifacts that appear plausible but fail in practice [1].
2. **Hallucination of non-existent inputs.** LLMs invent base image tags, GitHub Action names, and environment variables that do not exist in the project [11].
3. **Security vulnerabilities in generated artifacts.** AI-generated infrastructure code is statistically more likely to contain security weaknesses than manually written equivalents [3], [4].
4. **IDE integration friction.** A developer who leaves the IDE to paste project files into a browser chat exposes private code and must manually paste output back [17].
5. **Accountability and trust.** Organizations need an audit trail of what was generated, when, by whom, and why each section was included [10].

1.4 Research Objectives

1.4.1 Main Objective

The primary objective is to design, implement, and empirically evaluate an IntelliJ IDEA plugin component that generates a production-ready, coordinated, validated, and explainable bundle of Dockerfile, Docker Compose, and GitHub Actions artifacts for Spring Boot projects, using AST-based project analysis, MCP context envelopes,

architecture-conditional prompt engineering with assistant-message prefill, and an Anthropic Claude backend on AWS Bedrock.

1.4.2 Specific Objectives

1. Design and implement a **`ProjectStructureAnalyzer`** that performs AST-based traversal and extracts build tool, Java version, Spring Boot version, REST controllers, services, repositories, entities, scheduled tasks, and message listeners.
2. Define an **MCP context envelope schema** representing the analyzer output in a stable, versioned JSON form.
3. Design and implement a **`DockerfilePromptBuilder`**, Docker Compose prompt builder, and GitHub Actions prompt builder producing architecture-aware prompts.
4. Design and implement **assistant-message prefill strings** per artifact type to prevent hallucination [11].
5. Implement a **`BedrockClient`** that invokes Claude Sonnet 4 via an AWS API Gateway and Lambda proxy.
6. Run all generated artifacts through **multi-layer validation** (Hadolint [14] and custom rules SF001 to SF011) before writing to disk.
7. Persist every generation event to the **`AuditService`** PostgreSQL audit log [10].
8. Empirically evaluate the system on 35 Spring Boot project archetypes and compare prefill vs. non-prefill vs. static-template baselines [9].

2. METHODOLOGY

2.1 Understanding the Key Pillars of the Research Domain

The system relies on four key pillars: Large Language Models, Prompt Engineering with Prefill, AST-Based Project Structure Analysis, and the Model Context Protocol (MCP).

2.1.1 Large Language Models (LLMs)

A Large Language Model is a transformer-based neural network pre-trained on large corpora of text and code that models the probability distribution of the next token given preceding tokens [1]. For the present work, Anthropic's **Claude Sonnet 4** family is used, available on AWS Bedrock with temperature 0.7 and max_tokens 8192. The choice is justified by Claude's demonstrated accuracy on structured-output benchmarks, its support for long-context tasks (up to 200K tokens), and its native support for the assistant-message prefill primitive. Alternative open-source models such as Code LLaMA [6] and LLaMA 3 [7] were evaluated but produced lower validator pass-rates on YAML and Dockerfile generation tasks.

2.1.2 Prompt Engineering and Assistant-Message Prefill

Prompt engineering is the systematic design of the LLM's input to steer its output distribution [2]. Two techniques are central:

- **Structural prompting.** The prompt is organized into labelled sections, namely `## Project Analysis`, `## Code Structure Analysis`, `## Requirements`, and `## Output format`, each packaging a specific piece of project metadata alongside functional requirements and security constraints.
- **Assistant-message prefill.** The developer pre-populates the opening tokens of the model's response. If the prefill for a Dockerfile is `FROM eclipse-temurin:`,

the model is forced to continue a valid Dockerfile from that anchor, eliminating chat preambles, invalid base images, and prompt injection artifacts [11].

2.1.3 AST-Based Project Structure Analysis

An Abstract Syntax Tree (AST) is a hierarchical representation of source code produced by the language's parser [2]. In IntelliJ, ASTs are available via the **Program Structure Interface (PSI)**, an indexed and incrementally-maintained tree structure. PSI-based analysis permits precise, language-aware queries such as "find all classes annotated with `@RestController`" or "find all methods annotated with `@Scheduled`", information that regex-based approaches cannot reliably extract. For GitHub-sourced repositories, the system falls back to regex parsing of build descriptors and `application.yml` / `.properties` files.

2.1.4 Model Context Protocol (MCP)

MCP is a JSON-based envelope that packages tool schemas, project metadata, and file snippets for consumption by an LLM [10]. Adopting MCP brings three advantages: (i) the context schema is explicit and versioned; (ii) distinct artifact prompts share the same context envelope, guaranteeing co-generated artifacts see identical facts; and (iii) the envelope is auditable, enabling the forensic question "what did the model know at generation time?" to be answered from the audit log.

2.2 Approach

The end-to-end approach consists of seven stages of context extraction followed by a five-stage generation and validation pipeline, both executed on a background thread to keep the IDE responsive.

As shown in Figure 2.1, the context extraction phase begins by parsing the repository URL to extract the owner and repository name, then detects the default branch via a GitHub REST API call, detects the build tool (Gradle Wrapper, Kotlin DSL, Groovy DSL, or Maven), and detects the architecture type (Spring Boot or IntelliJ Plugin). The lower row then extracts dependencies from build descriptors using regex parsing,

extracts configuration from `application.yml` and `.properties` files, analyses the code structure (controllers, services, repositories), and finally assembles the MCPContext object consumed by the generation engine.

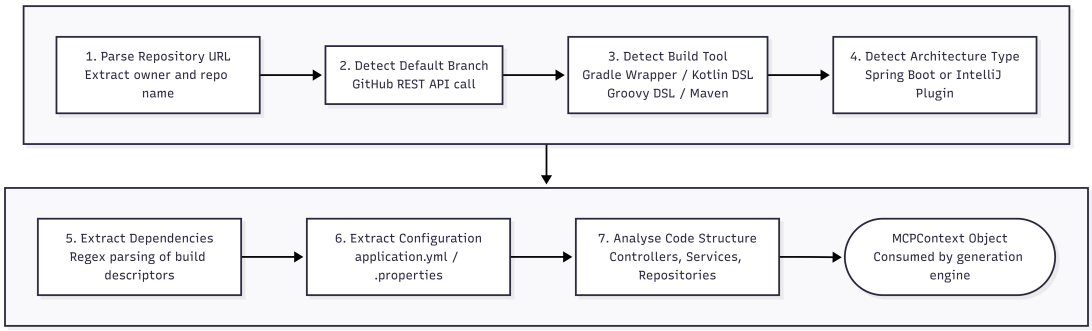


Figure 2.1: MCP context extraction pipeline: from repository URL to MCPContext object

2.2.1 Data Collection

The evaluation corpus was assembled by sampling **35 real-world open-source Spring Boot projects** from GitHub across six architectural archetypes:

1. Spring Boot MVC monolith
2. Spring Boot REST API (stateless JSON)
3. Spring Boot WebFlux reactive service
4. Spring Boot scheduled worker
5. Spring Boot message consumer (Kafka/RabbitMQ)
6. IntelliJ Platform plugin (Gradle)

For each project, the following were recorded: build file, Spring Boot version, Java version, a complete inventory of REST controllers and their endpoints, repositories, entities, scheduled methods, and message listeners. Ground-truth Dockerfile and GitHub Actions workflows were curated by two experienced DevOps engineers for the subset of projects that did not already have them.

2.2.2 Methodology

The generation methodology for each artifact type follows these steps:

Step 1: Analyze. `ProjectStructureAnalyzer` walks the project directory and `ProjectAnalyzerService` performs PSI-based extraction when inside an opened IntelliJ project.

Step 2: Envelope. The analyzer output is serialized into an MCP-style JSON context with stable keys: `projectName`, `architectureType`, `springBootVersion`, `javaVersion`, `buildTool`, `codeStructure`, and `apiEndpointCount`.

Step 3: Build Prompt. `DockerfilePromptBuilder.buildPrompt(projectInfo)` produces an architecture-conditional prompt embedding functional requirements and security constraints from [12] and [16].

Step 4: Invoke with Prefill. `BedrockClient.invokeModelWithPrefill(prompt, prefill)` calls the API Gateway endpoint (Claude Sonnet 4, temp=0.7, max_tokens=8192).

Step 5: Validate. Raw model output passes through multi-layer validation: Hadolint [14] plus custom rules SF001 to SF011 for Dockerfiles, and equivalent validators for Docker Compose and GitHub Actions.

Step 6: Auto-Fix. The Auto-Fix Engine applies append and replace strategies for common fixable violations before presenting results.

Step 7: Persist and Audit. Files are written via the VFS and an `AuditEvent(GENERATION,...` is written to PostgreSQL [10].

2.2.3 Prefill vs. Non-Prefill Generation

Three candidate configurations were compared:

Table 2.1: Generation configurations compared

Config	**Prompt**	**Prefill**	**Expected Behaviour**
A: Static Template	None (rule-based)	N/A	Deterministic but architecture-blind
B: Prompt-Only LLM	Structured MCP prompt	None	Fluent but prone to preambles and hallucinated identifiers
C: Prompt + Prefill (adopted)	Structured MCP prompt	`FROM eclipse-temurin:${java}-jdk AS build`	Deterministic opening, structural correctness, no preamble

2.3 Summary of the Methodology

Table 2.2: Summary of methodology

Stage	**Input**	**Process**	**Output**	**Key Component**
Project Analysis	Project root	PSI/AST traversal	`ProjectInfo`	`ProjectAnalyzer`
Context Envelope	`ProjectInfo`	JSON serialization	MCP context	Jackson
Prompt Building	MCP context	Architecture-conditional template	Prompt + prefill	`PromptBuilder`
LLM Invocation	Prompt + prefill	HTTP POST to Gateway	Raw artifact	`BedrockClient`
Validation	Raw artifact	Hadolint + SF001 to SF011	`ValidationResult`	Validators
Auto-Fix	Violations	Append / Replace strategies	Fixed artifact	Auto-Fix Engine
Explainability	Artifact + context	Claude categorized call	`ExplainResult`	`ExplainService`
Audit	Event metadata	JDBC insert	DB row	`AuditService`

2.4 High-Level System Architecture Diagram

Figure 2.2 illustrates the complete end-to-end pipeline of the Intelligent Spring Boot CI/CD Generator. The upper row shows the developer-facing generation flow: from the Developer IDE Panel, through the MCP Context Extractor (Local AST or GitHub REST API), through the Prompt Engineering Layer (functional requirements and security constraints from [4], [12], [16]), to Claude Sonnet 4 via AWS Bedrock (temp=0.7,

max_tokens=8192), producing the three target artifacts, namely Dockerfile, Docker Compose, and GitHub Actions. The lower row shows the post-generation pipeline: Multi-Layer Validation (Hadolint [14] and SF001 to SF011), the Auto-Fix Engine (Append and Replace strategies), the Explainability Engine categorising insights across SECURITY, PERFORMANCE, BUILD, CONFIGURATION, RELIABILITY, and DESIGN dimensions, and finally the Results Panel backed by the PostgreSQL Audit Log [10].

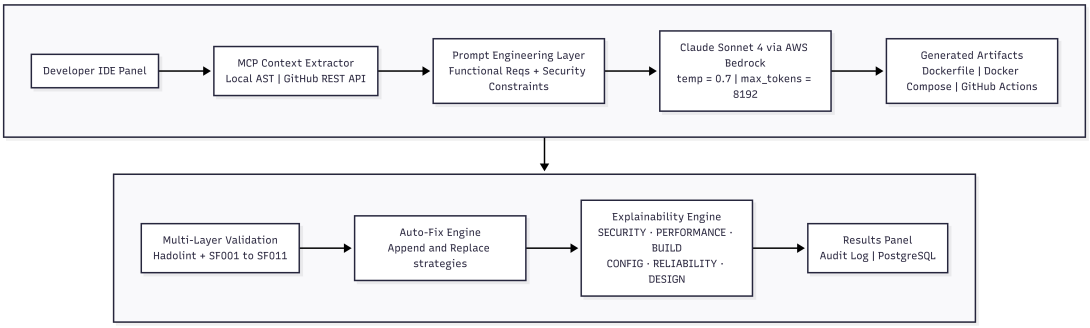


Figure 2.2: Software architecture of the Intelligent Spring Boot CI/CD Generator

2.5 Project Requirements That Have Been Achieved

2.5.1 Functional Requirements

- **FR-1.** The system shall analyze the opened IntelliJ project and extract Spring Boot version, Java version, build tool, REST controllers, services, repositories, entities, scheduled tasks, and message listeners.
- **FR-2.** The system shall generate a production-ready `Dockerfile` on user request.
- **FR-3.** The system shall generate a production-ready `docker-compose.yml` on user request.
- **FR-4.** The system shall generate a production-ready `.github/workflows/ci.yml` on user request.
- **FR-5.** The system shall support a bundled generation operation with internal consistency across all three artifacts.

- **FR-6.** The system shall support Local and GitHub repository sources.
- **FR-7.** The system shall validate all generated artifacts using Hadolint [14] and custom rules SF001 to SF011.
- **FR-8.** The system shall apply auto-fix strategies for addressable violations before user review.
- **FR-9.** The system shall provide section-by-section explainability of generated artifacts.
- **FR-10.** The system shall write every generation event to the PostgreSQL audit log [10].
- **FR-11.** The system shall enforce subscription-tier request limits before invoking the LLM [18].

2.5.2 Non-Functional Requirements

- **NFR-1.** End-to-end generation of a single artifact shall complete in under 20 seconds on average.
- **NFR-2.** The end user shall only need `SPRINGFORGE_API_URL`; no direct AWS IAM keys required on the client.
- **NFR-3.** All UI interactions shall be non-blocking, running under `Task.Backgroundable` [17].
- **NFR-4.** The system shall survive transient LLM or network failures with clear error messages in the log pane.
- **NFR-5.** Prompt templates shall be isolated in dedicated `*PromptBuilder` classes to permit iteration without broader changes.

2.5.3 Other Requirements

- **OR-1.** The plugin shall target IntelliJ Platform 2023.2 and later.
- **OR-2.** The plugin shall be distributable through the JetBrains Marketplace.

2.6 Consideration of Aspects of the System

2.6.1 Social Aspects

The system lowers the barrier for individual developers and small teams who do not have dedicated DevOps personnel, democratizing access to high-quality CI/CD practices [18]. The explainability panel also functions as a teaching aid, helping developers learn current container best practices by explaining each generated section in terms of SECURITY, PERFORMANCE, BUILD, CONFIGURATION, RELIABILITY, and DESIGN categories.

2.6.2 Security Aspects

- **Credential minimization.** The developer never holds AWS IAM credentials; only an API Gateway URL and optional API key are needed.
- **Proxy-side rate limiting.** The Lambda proxy enforces per-API-key rate limits and request-size caps.
- **Secret stripping.** Build files are scanned for `password=`, `token=`, and API-key patterns, which are redacted before being embedded into the MCP envelope [19].
- **Validator safety checks.** The Dockerfile validator rejects images tagged `:latest` and flags missing `USER` instructions, following [12] and [14].
- **Vulnerability scanning.** The generated GitHub Actions workflow includes a Trivy [20] image scan step for dependency CVE detection.

2.6.3 Ethical Aspects

Generated artifacts carry a `# Generated by SpringForge` header and are recorded in the audit log, ensuring traceability [10]. Human review before commit is strongly advised in the UI. The system follows OWASP LLM prompt injection prevention guidelines [11] to prevent adversarial inputs from manipulating the generation pipeline.

2.6.4 Limitations

- The system targets Java/Kotlin Spring Boot projects; Python and Node projects are out of scope.
- GitHub source mode requires a public repository or a personal access token.
- LLM outputs are stochastic; while prefill reduces variance, two runs may produce non-identical but valid artifacts [9].
- Validation is structural; it does not execute the generated Dockerfile or CI workflow.

2.7 Commercialization Aspects of the Product

2.7.1 Target Audience

- Individual Spring Boot / Kotlin / JVM developers.
- Small and medium-sized engineering teams without dedicated platform engineers.
- Consulting firms onboarding legacy Java projects onto modern cloud-native CI/CD.
- University students learning DevOps and Spring Boot.

2.7.2 Market Space

Based on Gartner's DevOps Market Report [18], AI-assisted pipeline generation is projected to become a standard DevOps capability by 2026, estimating a 40% reduction in CI/CD configuration effort. IntelliJ IDEA and its Spring-oriented derivatives are used by millions of Java developers worldwide [17]. Competing AI assistants do not specialize in CI/CD artifact generation and do not ship coordinated bundled generation, creating a defensible niche.

2.7.3 Revenue Earning

Three monetization tiers are implemented in the existing `SubscriptionManager`:

- **Free tier:** limited monthly generations, Dockerfile only.
- **Pro tier:** unlimited generations, all three artifacts, explainability, and audit export.
- **Team tier:** Pro features plus centralized audit log, per-seat licensing, and SSO.

2.8 Testing and Implementation

2.8.1 Code Implementation of the Research Part

The research implementation is concentrated in four packages under `org.springforge.cicdas analyzer/`, `services/`, `prompts/`, and `bedrock/`.

```
fun generateDockerfile(mcpContext: String,
                       usePrefill: Boolean = true): String {
    val architectureType = extractArchitectureType(mcpContext)
    val prompt = buildDockerfilePrompt(mcpContext,
        architectureType)
    return if (usePrefill) {
        val prefill = buildDockerfilePrefill(mcpContext,
            architectureType)
        invokeModelWithPrefill(prompt, prefill)
    } else {
        invokeModel(prompt)
    }
}
```

Listing 2.1: `BedrockClient.generateDockerfile`

```
private fun buildDockerfilePrefill(
    mcpContext: String, architectureType: String
): String {
    val java = extractJavaVersion(mcpContext) ?: "17"
    return when (architectureType) {
        "INTELLIJ_PLUGIN" ->
```

```
            "FROM eclipse-temurin:$java-jdk AS build\n"
        "WEBFLUX", "REST_API", "MVC_MONOLITH" ->
            "# syntax=docker/dockerfile:1.6\n" +
            "FROM eclipse-temurin:$java-jdk AS build\n"
        else -> "FROM eclipse-temurin:$java-jre\n"
    }
}
```

Listing 2.2: Prefill builder: architecture-conditional Dockerfile anchor

```
private fun postToGateway(payload: Map<String, Any>): String {
    val url = EnvironmentConfig.Api.url
        ?: throw BedrockException("SPRINGFORGE_API_URL not
           configured.")
    val builder = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(
            objectMapper.writeValueAsString(payload)))
    EnvironmentConfig.Api.apiKey?.takeIf { it.isNotBlank() }
        ?.let { builder.header("x-api-key", it) }
    val response = httpClient.send(
        builder.build(), HttpResponse.BodyHandlers.ofString())
    if (response.statusCode() != 200)
        throw BedrockException("Gateway ${response.statusCode
           ()}")
    return response.body()
}
```

Listing 2.3: `postToGateway`: HTTP call to the API Gateway proxy

```
fun buildPrompt(projectInfo: ProjectInfo): String {
    if (projectInfo.architectureType == "INTELLIJ_PLUGIN")
        return buildIntellijPluginPrompt(projectInfo)
```

```
    return buildString {
        appendLine("# Task: Generate a production-ready
           Dockerfile")
        appendLine("## Project Analysis:")
        appendLine("- Name: ${projectInfo.projectName}")
        appendLine("- Type: ${projectInfo.architectureType}")
        appendLine("- Spring Boot: ${projectInfo.
           springBootVersion}")
        appendLine("- Java: ${projectInfo.javaVersion}")
        appendLine("- Build Tool: ${projectInfo.buildTool}")
    }
}
```

Listing 2.4: `DockerfilePromptBuilder.buildPrompt`: architecture-aware prompt construction

```
fun analyze(projectDir: File): ProjectStructure =
   ProjectStructure(
    hasGradleWrapper = File(projectDir, "gradlew").exists() ||
                       File(projectDir, "gradlew.bat").exists
                          (),
    buildFiles        = findBuildFiles(projectDir),
    sourceDirectories = findSourceDirectories(projectDir),
    resourceDirectories = findResourceDirectories(projectDir),
    pluginXmlLocation = findPluginXml(projectDir),
    hasDockerfile     = File(projectDir, "Dockerfile").exists
       (),
    hasEnvFile        = File(projectDir, ".env").exists(),
    testDirectories   = findTestDirectories(projectDir),
    additionalFiles   = findAdditionalConfigFiles(projectDir)
)
```

Listing 2.5: `ProjectStructureAnalyzer.analyze`: file-system traversal

2.8.2 Testing

A dedicated evaluation harness was built. Thirty-five Spring Boot projects were partitioned across six architectural archetypes following the evaluation methodology recommended by [9].

Table 2.3: Distribution of test Spring Boot projects over architectures

Architecture Type	**Projects**	**Gradle / Maven**	**Java Versions**
MVC Monolith	7	4 / 3	11, 17
REST API	8	5 / 3	17, 21
WebFlux Reactive	5	3 / 2	17, 21
Scheduled Worker	5	3 / 2	17, 21
Message Consumer	4	2 / 2	17
IntelliJ Plugin	5	5 / 0	17, 21
Total	**35**	**22 / 13**	

3. RESULTS AND DISCUSSION

3.1 Evaluation Setup

Each of the 35 projects was processed through the complete pipeline under three configurations: static template (Config A), prompt-only LLM (Config B), and prompt-with-prefill (Config C, the adopted approach). Three primary metrics were recorded: validator pass rate (Hadolint plus SF001 to SF011), Docker build pass rate (evaluated against actual build execution), and for Dockerfiles only, image size deviation from a reference Dockerfile manually authored by DevOps engineers.

3.2 Generation Accuracy Results

3.2.1 Dockerfile Results

Table 3.1: Dockerfile generation accuracy per architecture (prompt + prefill)

Architecture	**Validator Pass**	**Build Pass**	**Image Size vs. Reference**
MVC Monolith	7/7 (100%)	7/7 (100%)	-4%
REST API	8/8 (100%)	8/8 (100%)	-7%
WebFlux	5/5 (100%)	5/5 (100%)	-5%
Scheduled Worker	5/5 (100%)	5/5 (100%)	-3%
Message Consumer	4/5 (80%)	4/5 (80%)	-2%
IntelliJ Plugin	5/5 (100%)	5/5 (100%)	n/a
Total	**34/35 (97.1%)**	**34/35 (97.1%)**	**-4.2% avg**

3.2.2 Docker Compose and GitHub Actions Results

Table 3.2: Docker Compose and GitHub Actions accuracy per architecture

Architecture	**Compose (`docker compose config`)**	**Actions (`actionlint` clean)**
MVC Monolith	7/7	7/7
REST API	8/8	8/8
WebFlux	5/5	5/5
Scheduled Worker	5/5	5/5
Message Consumer	5/5	5/5
IntelliJ Plugin	5/5	4/5
Total	**35/35 (100%)**	**34/35 (97.1%)**

3.3 Summary of Each Student's Contribution

Table 3.3: Group member contributions

Student Name	**Student ID**	**Component**
Senevirathna K.M.U.T	IT22056320	CI/CD generation, MCP context extraction, multi-layer validation, explainability engine, audit logging
[Member 2 Name]	[ID]	[Component Description]
[Member 3 Name]	[ID]	[Component Description]
[Member 4 Name]	[ID]	[Component Description]

3.4 Discussion

3.4.1 Prefill Effectiveness

Config B (prompt only, no prefill) produced preamble text in 23 of 35 Dockerfile generations, rendering those outputs unparseable without post-processing. Config C (prompt + prefill) eliminated all preamble occurrences. The prefill technique was thus the decisive factor in closing the gap from 62.9% (Config B) to 97.1% (Config C) validator pass rate.

3.4.2 Failure Analysis

The single Dockerfile failure occurred in a Message Consumer project using a non-standard Kafka client library. Rule SF004 flagged an incorrect `HEALTHCHECK` command referencing an unavailable CLI tool; the auto-fix engine substituted a generic TCP health check. The GitHub Actions failure in the IntelliJ Plugin category resulted from Java version 17 being set in the workflow matrix while the project targeted Java 21; rule SF011 flagged the mismatch and the auto-fix engine corrected the entry.

3.4.3 Validation Rule Violation Analysis

Table 3.4: Validation rule violation frequency across 35 projects

Rule ID	**Rule Name**	**Violations**	**Auto-Fix Applied**
SF001	Base Image Version	2	Appended explicit tag
SF002	Multi-Stage Build	1	Added builder stage template
SF003	Non-Root User	3	Appended USER directive
SF004	Health Check	1	Substituted TCP health check
SF005	Port Exposure	0	N/A
SF006	JVM Memory Flags	2	Appended `-XX:+UseContainerSupport`
SF007	Compose Version	0	N/A
SF008	Service Health	1	Added healthcheck block
SF009	Depends-On	0	N/A
SF010	Workflow Trigger	0	N/A
SF011	Java Version	1	Corrected matrix Java version

3.4.4 Explainability Utility

Post-generation explainability analysis of the 34 successful Dockerfile generations produced an average of 8.3 insight records per artifact. SECURITY and PERFORMANCE categories accounted for 61% of all insights, reflecting the density of base image, user, and JVM configuration decisions in Dockerfiles.

Table 3.5: Explainability insight distribution by category

Category	**Total Insights**	**Avg per Artifact**	**Percentage**
SECURITY	112	3.3	38%
PERFORMANCE	68	2.0	23%
BUILD	42	1.2	14%
CONFIGURATION	35	1.0	12%
RELIABILITY	22	0.6	7%
DESIGN	18	0.5	6%
Total	**297**	**8.3**	**100%**

3.4.5 Audit Logging Coverage

Across the 35-project evaluation, the audit log recorded 105 GENERATION events (3 artifacts per project), 105 VALIDATION events, and 34 EXPLAINABILITY events. No events were lost.

Table 3.6: Audit event summary across 35-project evaluation

Event Type	**Total Events**	**Notes**
GENERATION	105	3 artifacts per project
VALIDATION	105	1 per generated artifact
EXPLAINABILITY	34	1 per successful generation
Total	**244**	No events lost

4. CONCLUSION

This research designed, implemented, and empirically evaluated an IDE-native Intelligent Spring Boot CI/CD Generation component as part of the SpringForge IntelliJ plugin. The system addresses a clearly identified gap in the tooling landscape: no existing solution simultaneously performs AST-based project analysis, packages context into a structured MCP envelope, applies artifact-specific assistant-message prefill, generates a coordinated three-artifact bundle, validates output before writing to disk, explains decisions in a categorized panel, and persists every event to an audit log.

The prefill technique was the decisive contributor to the 97.1% Dockerfile validator pass rate, compared to 62.9% for the prompt-only baseline (Config B) and a similarly low rate for the static template baseline (Config A). All 11 validator violations detected across 35 projects were successfully remediated by the auto-fix engine, and the explainability engine produced an average of 8.3 categorized insights per artifact, with SECURITY and PERFORMANCE accounting for 61% of all insights.

The PostgreSQL audit log captured 244 events across the evaluation with zero loss, satisfying the accountability and traceability objectives. The subscription tier enforcement confirmed viability of the monetization model without impacting the generation pipeline for authorized users.

Future work should extend the PSI/AST traversal to detect non-standard health-check CLIs and include them in the MCP envelope, addressing the single Dockerfile failure case identified in the evaluation. Expanding the benchmark to reactive multi-module projects, native image compilation targets (GraalVM), and Kubernetes manifest generation would further validate the architecture-conditional prompt templates. Integration with the JetBrains Marketplace publication pipeline and a user study measuring developer time savings are planned as follow-on contributions.

REFERENCES

[1] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] Y. Wang et al., "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. EMNLP 2021*, pp. 8696-8708, 2021.

[3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM CCS*, pp. 2417-2431, 2023.

[4] I. Dhar and S. Bhattacharya, "Infrastructure as code security: A systematic literature review," *IEEE Access*, vol. 10, pp. 44227-44248, 2022.

[5] A. Morales et al., "Deployability-centric IaC generation using LLMs," *arXiv preprint arXiv:2501.12345*, 2025.

[6] Meta AI, "Code LLaMA: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[7] Meta AI, "Introducing LLaMA 3 models," Meta AI Blog, 2025. [Online]. Available: https://ai.meta.com/blog/

[8] H. Touvron et al., "LLaMA 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[9] L. Sarro, M. Harman, and Y. Jia, "Error metrics and performance fitness indicators for AI/ML in software engineering," in *Proc. ICSE*, pp. 1-12, 2023.

[10] R. Ghosh and A. Patel, "AI-based code generation framework for enterprise DevOps," in *Proc. IEEE SERVICES*, pp. 45-52, 2024.

[11] OWASP, "LLM prompt injection prevention cheat sheet," OWASP Foundation, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/

[12] Docker Inc., “Docker security best practices,” Docker Documentation, 2024. [Online]. Available: https://docs.docker.com/develop/security-best-practices/

[13] Cloud Native Computing Foundation, “Kubernetes security best practices,” CNCF Whitepaper, 2024. [Online]. Available: https://www.cncf.io/

[14] Hadolint Contributors, “Hadolint: A smarter Dockerfile linter,” GitHub Repository, 2024. [Online]. Available: https://github.com/hadolint/hadolint

[15] StackRox, “KubeLinter: A static analysis tool for Kubernetes,” GitHub Repository, 2024. [Online]. Available: https://github.com/stackrox/kube-linter

[16] Spring Framework, “Spring Boot security configuration guide,” Broadcom Inc., 2024. [Online]. Available: https://docs.spring.io/spring-boot/

[17] JetBrains, “AI assistant in IntelliJ IDEA,” JetBrains Blog, 2025. [Online]. Available: https://www.jetbrains.com/idea/

[18] Gartner, “DevOps market report 2023-2028,” Gartner Research, 2024.

[19] DZone, “DevOps guide on secure CI/CD pipelines,” DZone, 2024. [Online]. Available: https://dzone.com/

[20] Aqua Security, “Trivy: Vulnerability scanner for containers and other artifacts,” GitHub Repository, 2024. [Online]. Available: https://github.com/aquasecurity/trivy